

4

Python Loops

- while & for loops
 - range() function
 - break, continue & pass statements
- 
- Two overlapping circles in shades of blue are located in the bottom right corner of the slide. The larger circle is a medium blue, and the smaller one is a lighter blue, partially overlapping the larger one.

What are Loops?

- A loop is used to execute a block of code repeatedly until a condition becomes false or all items in a sequence have been processed.

Why do we use loops?

- To avoid writing the same code again and again.
- To automate repetitive tasks.
- To work with lists, strings, tuples, dictionaries, etc.
- To make programs shorter and more efficient.

1. while Loop — Definition & Syntax

A while loop keeps executing as long as the condition is True.

 syntax

```
1 while condition:
2     # Code
```

while Loop — Example 1

main.py

```
1 i = 1
2 while i <= 5:
3     print(i)
4     i += 1
```

Output:

- 1
- 2
- 3
- 4
- 5

while Loop — Example 2: Password Check

main.py

```
1 password = ""
2 while password != "python":
3     password = input("Enter Password: ")
4 print("Access Granted")
```

Note:

The loop keeps asking for input until the correct password ("python") is entered — then it prints "Access Granted".



Advantages of while Loop

- Best when the number of iterations is unknown.
- Useful for user input.
- Useful for menus and games.

2. for Loop — Definition & Syntax

A for loop is used to iterate over a sequence such as a string, list, tuple, set, dictionary, or range.

syntax

```
1  for variable in sequence:  
2      # Code to execute
```

for Loop — Example 1: Print Names

```
main.py  
  
1 names = ["Ali", "Ahmed", "Sara"]  
2 for name in names:  
3     print(name)
```

Output:

- Ali
- Ahmed
- Sara

for Loop — Example 2: Print Characters

main.py

```
1 word = "Python"
2 for letter in word:
3     print(letter)
```

Output:

- P
- y
- t
- h
- o
- n



Advantages of for Loop

- Easy to read.
- Best for iterating over collections.
- Automatically stops when data ends.
- Less chance of infinite loops.

Difference Between for and while Loop

Feature	for Loop	while Loop
Used for	Known number of iterations	Unknown number of iterations
Condition	Automatically handled	Must be written manually
Infinite Loop Risk	Low	High (if condition never becomes False)
Best For	Lists, Strings, Range	User input, menus, games

range() Function in Python

What is range()?

- The range() function is a built-in Python function that generates a sequence of numbers.
- It is most commonly used with the for loop to repeat a block of code a specific number of times.

Why do we use range()?

- To generate a sequence of numbers.
- To control how many times a loop runs.
- To specify a starting number, ending number, and step size.
- To avoid writing numbers manually.

range() — Syntax & Parameters

syntax

```
1 range(stop)
2 range(start, stop)
3 range(start, stop, step)
```

Parameter	Description
start	Starting value (default is 0)
stop	Ending value (not included)
step	Difference between each number (default is 1)

Note: The stop value is never included in the output.

1. range(stop)

Starts from 0 and ends before the given stop value.

```
main.py  
  
1  for i in range(5):  
2      print(i)
```

Output:

- 0
- 1
- 2
- 3
- 4

2. range(start, stop)

Starts from the specified start value and ends before the stop value.

```
main.py  
  
1 for i in range(2, 7):  
2     print(i)
```

Output:

- 2
- 3
- 4
- 5
- 6

3. range(start, stop, step) — Positive Step

Starts from start, ends before stop, and increases by step.

main.py

```
1 for i in range(2, 11, 2):  
2     print(i)
```

Output:

- 2
- 4
- 6
- 8
- 10

3. range(start, stop, step) — Negative Step

A negative step counts backward from start to stop.

```
main.py  
  
1 for i in range(10, 0, -2):  
2     print(i)
```

Output:

- 10
- 8
- 6
- 4
- 2

Converting range() to a List

Since range() returns a range object, you can convert it into a list.

```
main.py  
  
1 numbers = list(range(1, 6))  
2 print(numbers)
```

Output:

```
[1, 2, 3, 4, 5]
```

Practical Examples with range() (1/2)

Example 1: Print Numbers 1 to 10

```
main.py

1 for i in range(1, 11):
2     print(i)
```

Note:

Prints every whole number from 1 through 10.

Example 2: Print Even Numbers

```
main.py

1 for i in range(2, 21, 2):
2     print(i)
```

Note:

Prints even numbers from 2 through 20.

Practical Examples with range() (2/2)

Example 3: Print Odd Numbers

```
main.py

1 for i in range(1, 20, 2):
2     print(i)
```

Note:

Prints odd numbers from 1 through 19.

Example 4: Countdown

```
main.py

1 for i in range(5, 0, -1):
2     print(i)
3 print("Blast Off!")
```

Output:

- 5
- 4
- 3
- 2
- 1
- Blast Off!



Advantages of range()

- Generates numbers automatically.
- Saves time and reduces code repetition.
- Makes loops simple and readable.
- Supports both forward and backward counting.
- Allows custom step values.

Common Mistakes — Stop Is Not Included

Mistake: assuming the stop value is included in the output.

```
main.py  
  
1 for i in range(1, 5):  
2     print(i)
```

Output:

- 1
- 2
- 3
- 4

Note: 5 is not printed because the stop value is excluded.

Common Mistakes — Step Cannot Be Zero

Mistake: using a step value of 0.

 main.py

```
1 range(1, 10, 0)
```

Output:

ValueError: range() arg 3 must not be zero

range() — Summary Table

Syntax	Output
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(1, 5)</code>	1, 2, 3, 4
<code>range(2, 11, 2)</code>	2, 4, 6, 8, 10
<code>range(10, 0, -2)</code>	10, 8, 6, 4, 2

3. break — What It Does

- The break statement immediately exits the loop.

Why do we use break?

- Stop searching when an item is found.
- Exit the loop early.
- Save execution time.

break — Example

Stops the loop as soon as i equals 6.

```
main.py

1  for i in range(1, 11):
2      if i == 6:
3          break
4      print(i)
```

Output:

- 1
- 2
- 3
- 4
- 5

break — Example with while

An infinite while loop that exits using break.

```
main.py

1  i = 1
2  while True:
3      print(i)
4      if i == 5:
5          break
6      i += 1
```

Output:

- 1
- 2
- 3
- 4
- 5

4. continue — What It Does

- The continue statement skips the current iteration and moves to the next iteration.

Why do we use continue?

- Skip unwanted values.
- Ignore specific conditions.
- Continue processing remaining data.

continue — Example 1

Skips printing when i equals 3.

```
main.py

1  for i in range(1, 6):
2      if i == 3:
3          continue
4      print(i)
```

Output:

- 1
- 2
- 4
- 5

continue — Example 2

Skips the letter "h" while printing the rest.

```
main.py

1 word = "Python"
2 for letter in word:
3     if letter == "h":
4         continue
5     print(letter)
```

Output:

- P
- y
- t
- o
- n

5. pass — What It Does

- The pass statement is a placeholder. It does nothing but prevents an error when a statement is required.

Why do we use pass?

- Write incomplete code.
- Create empty functions or loops.
- Reserve code for future implementation.

pass — Example 1

pass does nothing — the loop continues normally.

```
main.py

1  for i in range(5):
2      if i == 3:
3          pass
4      print(i)
```

Output:

- 0
- 1
- 2
- 3
- 4

pass — Example 2 & 3

main.py

```
1 def student():  
2     pass
```

Note:

Creates an empty function without raising an error.

main.py

```
1 if True:  
2     pass  
3 else:  
4     print("False")
```

Note:

Nothing is printed — pass only satisfies Python's syntax.

Difference Between break, continue & pass

Statement	Purpose	Loop Continues?
break	Exits the loop immediately	✗ No
continue	Skips the current iteration	✓ Yes
pass	Does nothing (placeholder)	✓ Yes

while Loop (1/3) — Multiple Choice Questions

1. A while loop keeps executing as long as:

- A) The condition is False B) The condition is True C) i equals 0 D) The list is empty

2. What is the main risk of a while loop?

- A) Syntax error B) Type error C) Infinite loop D) Index error

3. Which loop suits an unknown number of iterations?

- A) for loop B) while loop C) nested loop D) range loop

4. What must usually be updated manually to avoid an infinite loop?

- A) The output B) The condition variable C) The function name D) The print statement

5. Which scenario best suits a while loop?

- A) Iterating a fixed list B) Printing 1 to 10 C) Waiting for correct input D) Looping string characters

Answer Key: 1-B 2-C 3-B 4-B 5-C

for Loop (2/3) — Multiple Choice Questions

1. Which keyword is used to iterate over a sequence in Python?

A) loop B) for C) each D) iterate

2. Which of these can a for loop iterate over?

A) Only lists B) Only strings C) Lists, strings, tuples, dicts D) Only numbers

3. What happens when a for loop finishes iterating all items?

A) Raises an error B) Restarts automatically C) Stops automatically D) Waits for input

4. In for letter in "Python":, what does letter represent?

A) The whole string B) A single character C) The string length D) A random letter

5. Compared to while, a for loop is generally:

A) Harder to read B) Riskier for infinite loops C) Easier for known collections D) Only for numbers

Answer Key: 1-B 2-C 3-C 4-B 5-C

range(), pass, continue & break (3/3) — Mixed Questions

1. What does range(1, 5) generate?

A) 1, 2, 3, 4, 5 B) 1, 2, 3, 4 C) 0, 1, 2, 3, 4 D) 2, 3, 4, 5

2. Which value is never included in a range() output?

A) start B) step C) stop D) None of these

3. Which statement is only a placeholder and does nothing?

A) break B) continue C) pass D) return

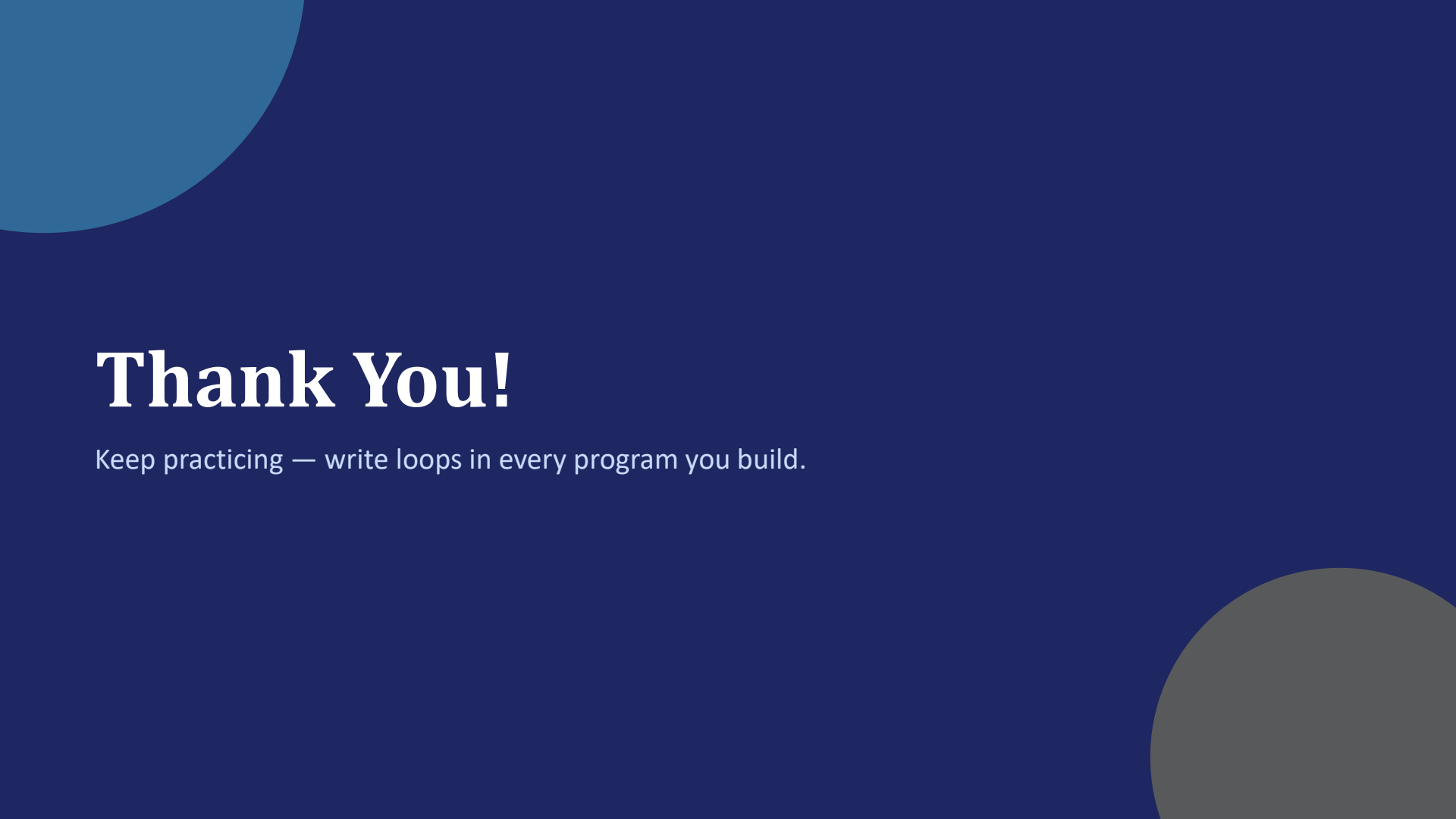
4. Which statement skips the current iteration and moves to the next?

A) break B) continue C) pass D) exit

5. Which statement immediately exits a loop?

A) continue B) pass C) break D) return

Answer Key: 1-B 2-C 3-C 4-B 5-C



Thank You!

Keep practicing — write loops in every program you build.